Sprawozdanie z projektu końcowego

```
#include <targets\AT91SAM7.h>
#include "pcf8833u8_lcd.h"
#include <stdint.h>
#include <string.h>
#define LCD_BACKLIGHT PIOB_SODR_P20 //lub (1<<20)
#define AUDIO_OUT PIOB_SODR_P19
#define AUDIO_CLR PIOB_CODR_P19

#define SW1 PIOB_SODR_P24
#define SW2 PIOB_SODR_P25

#define JOY_UP PIOA_SODR_P9
#define JOY_ENTER PIOA_SODR_P15
#define JOY_LEFT PIOA_SODR_P7
#define JOY_DOWN PIOA_SODR_P8
#define JOY_RIGHT PIOA_SODR_P14

#define SCREEN_WIDTH 130
#define SCREEN_HEIGHT 130
#define BG_COLOR WHITE

#include "pitches.h"
#define REST 0
int melody1[] = {
  NOTE_C4, NOTE_G3, NOTE_G3, NOTE_A3,
  NOTE_G3, 0, NOTE_B3, NOTE_C4
};

int noteDurations[] = {
  4, 8, 8, 4, 4, 4, 4, 4
};
int loseMelody[] = {
    NOTE_E4, NOTE_D4, NOTE_C4, NOTE_A3,
    NOTE_G3, NOTE_E3, NOTE_C3, REST
};
int loseDurations[] = {
    4, 4, 4, 4,
    2, 2, 1, 4
};

#define JOY_UP_PRESSED     PIOA_SODR_P9
#define JOY_ENTER_PRESSED  PIOA_SODR_P15
#define JOY_DOWN_PRESSED   PIOA_SODR_P8

#define JOY_RIGHT_PRESSED  PIOA_SODR_P14
#define JOY_LEFT_PRESSED   PIOA_SODR_P7
#define SW1_PRESSED  PIOB_SODR_P24
#define SW2_PRESSED  PIOB_SODR_P25



#define WALL_COLOR BLACK
#define PATH_COLOR WHITE
#define MAZE_SIZE 16
#define CELL_SIZE 6
#define LCD_SIZE 130
int endX = MAZE_SIZE - 1;
int endY = MAZE_SIZE - 1;
#define MAZE_PIXEL_SIZE (MAZE_SIZE * CELL_SIZE)
#define OFFSET ((LCD_SIZE - MAZE_PIXEL_SIZE) / 2)
```

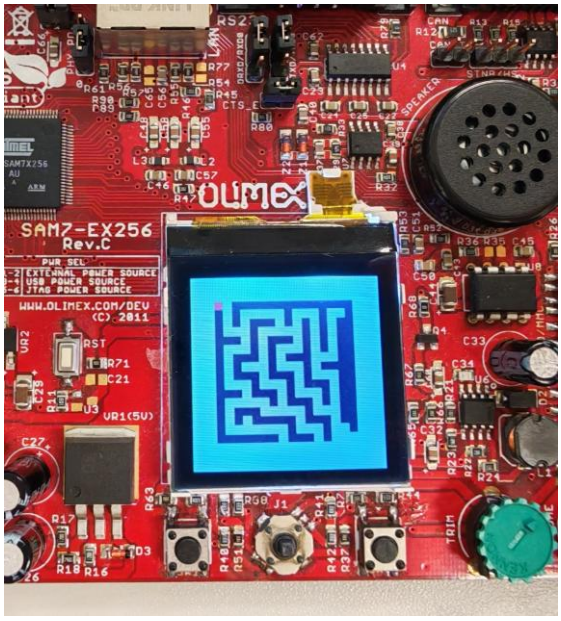
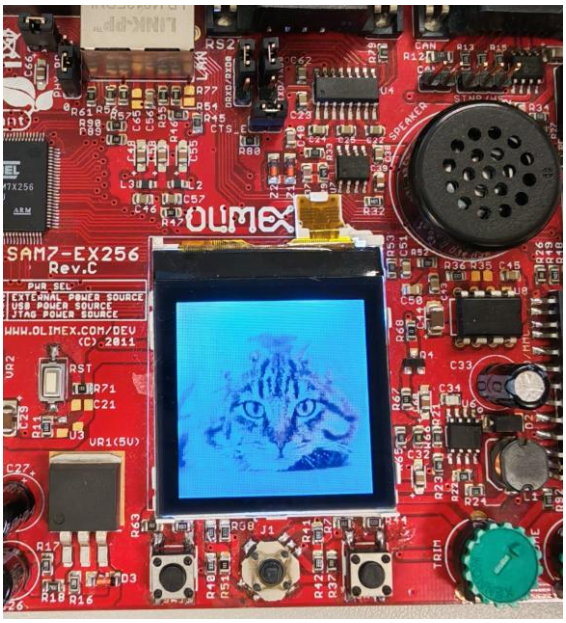
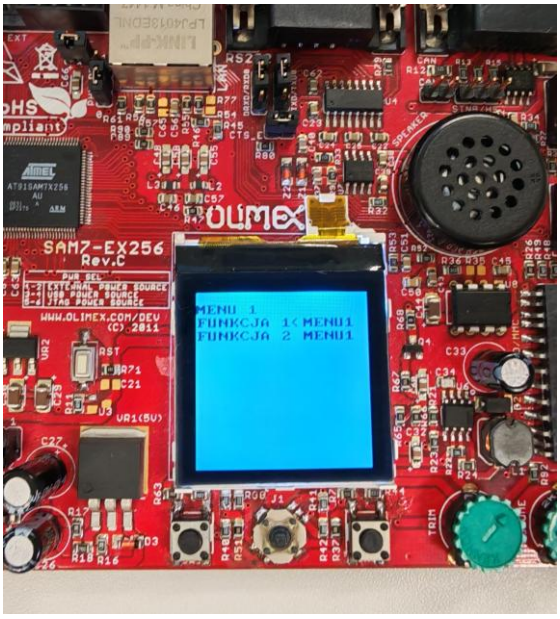
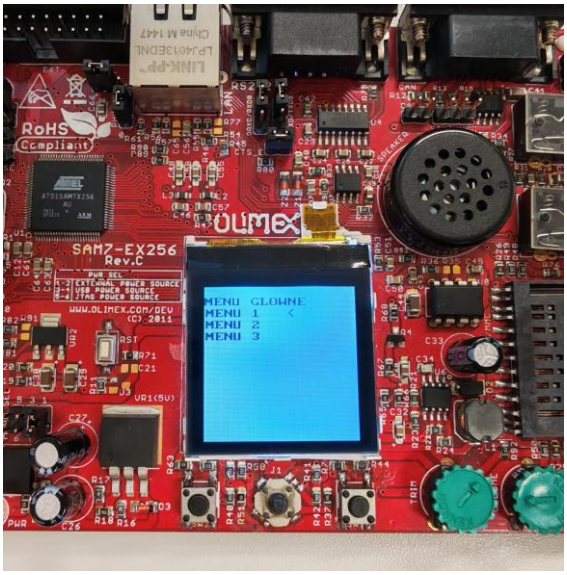
```
uint8_t maze[MAZE_SIZE][MAZE_SIZE] = {
    {0,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
    {0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,1},
    {1,1,1,0,1,0,1,1,1,0,1,0,1,1,0,1},
    {1,0,0,0,0,0,1,0,1,0,0,0,1,0,0,1},
    {1,0,1,1,1,1,1,0,1,1,1,0,1,1,0,1},
    {1,0,1,0,0,0,0,0,0,0,1,0,0,1,0,1},
    {1,0,1,0,1,1,1,1,1,0,1,1,0,1,0,1},
    {1,0,0,0,1,0,0,0,1,0,0,0,0,1,0,0},
    {1,1,1,0,1,0,1,0,1,1,1,1,0,1,1,1},
    {1,0,0,0,0,0,1,0,0,0,0,1,0,0,0,1},
    {1,0,1,1,1,1,1,1,1,1,0,1,1,1,0,1},
    {1,0,0,0,0,0,0,0,0,1,0,0,0,1,0,0},
    {1,1,1,1,1,1,0,1,0,1,1,1,0,1,1,1},
    {1,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0},
    {1,0,1,1,1,1,1,1,1,1,1,1,1,1,0,0},
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,0}
};

void delay_ms(uint32_t ms)
{
  volatile uint32_t i;
  while (ms--)
  for (i = 0; i < 3000; i++);
  }
  static inline uint8_t isPressed(uint32_t pin)
  {
  return !(PIOA_PDSR & pin);
  }
  static inline uint8_t isPressedSW(uint32_t pin)
  {
  return !(PIOB_PDSR & pin);
}

void playTone(uint32_t freq, uint32_t duration_ms)
{
    if (freq == 0)
    {
        delay_ms(duration_ms);
        return;
    }

    uint32_t period_us = 1000000 / freq;
    uint32_t cycles = (duration_ms * 1000) / period_us;

    for (uint32_t i = 0; i < cycles; i++)
    {
        PIOB_SODR = AUDIO_OUT;
        delay(1);
        PIOB_CODR = AUDIO_OUT;
        delay(1);
    }
}

void delay(int n) __attribute__ ((section(".fast")));
void delay(int n) //procedura op nienia
{
    volatile int i;
    for(i = 3000 * n; i > 0; i--)
    {
        __asm__("nop");
    }
}
```

```
void obrazek1(void)
{
    LCDClearScreen();
    LCDWrite130x130bmp();
    while (!isPressedSW(SW2));
    delay_ms(200);
    LCDClearScreen();
}

void obrazek2(void)
{
    LCDClearScreen();
    LCDWrite130x130bmp2();
    while (!isPressedSW(SW2));
    delay_ms(200);
    LCDClearScreen();
}

void playMelody1(void)
{
    LCDClearScreen();
    LCDPutStr("MELODY 1", 10, 10, MEDIUM, BLACK, WHITE);

    for (int i = 0; i < 8; i++)
    {
        int noteDuration = 1000 / noteDurations[i];
        playTone(melody1[i], noteDuration);

        delay_ms(noteDuration * 0.3);
    }

    while (!isPressedSW(SW2));

    delay_ms(200);
    LCDClearScreen();
}


void playLoseGifAndMusic(void)
{
    LCDClearScreen();
    LCDPutStr("MELODY 2", 10, 10, MEDIUM, BLACK, WHITE);
    for (int i = 0; i < 8; i++)
    {
        int noteDuration = 1200 / loseDurations[i];
        playTone(loseMelody[i], noteDuration);
        delay_ms(noteDuration * 0.3);
    }
    while (!isPressedSW(SW2));
    delay_ms(200);
    LCDClearScreen();

}
void drawCell(int x, int y, uint16_t color)
{
    int px = OFFSET + x * CELL_SIZE;
    int py = OFFSET + y * CELL_SIZE;

    for(int i=0; i<CELL_SIZE; i++)
        for(int j=0; j<CELL_SIZE; j++)
            LCDSetPixel(px + i, py + j, color);
}
```

```c
void drawMaze(void)
{
    for(int y=0; y<MAZE_SIZE; y++)
        for(int x=0; x<MAZE_SIZE; x++)
            drawCell(x, y, maze[y][x] ? BLACK : WHITE);
}
int playerX = 0;
int playerY = 0;

void drawPlayer(void)
{
    drawCell(playerX, playerY, RED);
}
void movePlayer(int dx, int dy)
{
    int nx = playerX + dx;
    int ny = playerY + dy;

    if(nx < 0 || nx >= MAZE_SIZE || ny < 0 || ny >= MAZE_SIZE)
        return; // outside maze

    if(maze[ny][nx] == 0)
    {
        drawCell(playerX, playerY, WHITE);
        playerX = nx;
        playerY = ny;
        drawPlayer();
    }
}
void mazeJoystickHandler(void)
{
    if(isPressed(JOY_UP_PRESSED)) movePlayer(-1, 0);
    if(isPressed(JOY_DOWN_PRESSED)) movePlayer(1, 0);
    if(isPressed(JOY_LEFT_PRESSED)) movePlayer(0, -1);
    if(isPressed(JOY_RIGHT_PRESSED)) movePlayer(0, 1);
}
void programMaze(void)
{
    LCDClearScreen();
    drawMaze();
    playerX = 0;
    playerY = 0;
    drawPlayer();

    while(1)
    {
      if(playerX == endX && playerY == endY)
      {
          LCDPutStr("YOU WIN!", OFFSET + 20, OFFSET + 20, MEDIUM, BLACK, WHITE);
      }
        mazeJoystickHandler();

        if(isPressedSW(SW2_PRESSED))
        {
            LCDClearScreen();
            return;
        }
        if(playerX == MAZE_SIZE-1 && playerY == MAZE_SIZE-1)
            LCDPutStr("YOU WIN!", OFFSET+20, OFFSET+20, MEDIUM, BLACK, WHITE);

        delay_ms(150);
    }
}
```

```c
typedef struct menu_struct menu_t;
struct menu_struct {
const char * name;
menu_t * next; // Nast pny element
menu_t * prev; // Poprzeni element
menu_t * child; // Dziecko danego wska nika
menu_t * parent; // Rodzic danego wska nika
void( * menu_function)(void);
};

menu_t menu_1, menu_2, menu_3, sub_menu_1_1, sub_menu_1_2,
sub_menu_2_1, sub_menu_2_2, sub_menu_3_1, sub_menu_3_2;

menu_t menu_1 = {
  "MENU 1 ",
  & menu_2,
  & menu_1,
  & sub_menu_1_1,
  NULL,
  NULL
};

menu_t sub_menu_1_1 = {
  "FUNKCJA 1 MENU1",
  & sub_menu_1_2,
  & sub_menu_1_1,
  NULL,
  & menu_1,
  obrazek1
};

menu_t sub_menu_1_2 = {
  "FUNKCJA 2 MENU1",
  NULL,
  & sub_menu_1_1,
  NULL,
  & menu_1,
  obrazek2
};

menu_t menu_2 = {
  "MENU 2 ",
  & menu_3,
  & menu_1,
  & sub_menu_2_1,
  NULL,
  NULL
};

menu_t sub_menu_2_1 = {
  "FUNKCJA 1 MENU2",
  & sub_menu_2_2,
  & sub_menu_2_1,
  NULL,
  & menu_2,
  playMelody1
};
```

```
menu_t sub_menu_2_2 = {
  "FUNKCJA 2 MENU2",
  NULL,
  & sub_menu_2_1,
  NULL,
  & menu_2,
  playLoseGifAndMusic
};

menu_t menu_3 = {
  "MENU 3 ",
  NULL,
  & menu_2,
  & sub_menu_3_1,
  NULL,
  NULL
};

  menu_t sub_menu_3_1 = {
  "FUNKCJA 1 ",
  & sub_menu_3_2,
  & sub_menu_3_1,
  NULL,
  & menu_3,
  programMaze
};

menu_t sub_menu_3_2 = {
  "MENU 1 ",
  NULL,
  & sub_menu_3_1,
  & sub_menu_1_1,
  & menu_3,
  NULL
};

menu_t *currentPointer = &menu_1;
uint8_t menu_index;
uint8_t lcd_row_pos;
void drawMenu(menu_t *ptr)
{
menu_t *temp;
uint8_t row = 0;

if (ptr->parent != NULL)
LCDPutStr(ptr->parent->name, 10, 0, MEDIUM, BLACK, WHITE);
else
LCDPutStr("MENU GLOWNE", 10, 0, MEDIUM, BLACK, WHITE);

temp = ptr;
while (temp->prev != temp && temp->prev != NULL)
temp = temp->prev;


while (temp != NULL && row < 3)
{
LCDPutStr(temp->name, row*10+20, 1, MEDIUM, BLACK, WHITE);
if (temp == ptr)
LCDPutStr( "<", row*10+20, 72, MEDIUM, BLACK, WHITE);
temp = temp->next;
row++;
}
}
```

```c
void menuJoystickUp(void)
{
if (currentPointer->prev != NULL)
currentPointer = currentPointer->prev;
}


void menuJoystickDown(void)
{
if (currentPointer->next != NULL)
currentPointer = currentPointer->next;
}


void menuEnter(void)
{
    if (currentPointer->child != NULL)
    {
        currentPointer = currentPointer->child;
    }
    else if (currentPointer->menu_function != NULL)
{

    currentPointer->menu_function();
}
}
void menuBack(void)
{
if (currentPointer->parent != NULL)
currentPointer = currentPointer->parent;
}

int main(void)
{
PMC_PCER = PMC_PCER_PIOB;
PMC_PCER = PMC_PCER_PIOA;
PIOB_OER = LCD_BACKLIGHT | AUDIO_OUT;
PIOB_PER = LCD_BACKLIGHT | AUDIO_OUT;
PIOA_PER = JOY_UP | JOY_DOWN | JOY_ENTER | SW1 | SW2;
PIOA_ODR = JOY_UP | JOY_DOWN | JOY_ENTER | SW1 | SW2;

InitLCD();
LCDSettings();
LCDClearScreen();

while (1)
{
if (isPressed(JOY_UP))
{
LCDClearScreen();
menuJoystickUp();
delay_ms(200);
}

if (isPressed(JOY_DOWN))
{
LCDClearScreen();
menuJoystickDown();
delay_ms(200);
}
```

```
if (isPressedSW(SW1))
{
LCDClearScreen();
menuEnter();
delay_ms(200);
}
if (isPressedSW(SW2))
{
LCDClearScreen();
menuBack();
delay_ms(200);
}

drawMenu(currentPointer);
}
}
```